

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME: Cloud Computing and
Big Data Analytics

COURSE CODE: CSA1522

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.*
(BL4)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 30

Code of Conduct

I Shri Shanmuga Priya K (192324035) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Assessment Tasks

Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.
2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.
3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.
4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed

messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.

5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

Reverse Engineering Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Inference Accuracy	25%	Infers system components and flow with high accuracy	Mostly accurate inference	Basic inference	Weak inference
Understanding of Architecture	25%	Shows clear understanding of underlying big data architecture	Good understanding	Partial understanding	Poor understanding
Use of Technical Evidence	20%	Uses strong reasoning and technical evidence	Reasonable evidence	Limited evidence	No meaningful evidence
Depth of Analysis	15%	Analysis is deep and insightful	Reasonably deep	Basic depth	Superficial analysis
Clarity	15%	Clear and organized explanation	Generally clear	Somewhat unclear	Poorly presented

1. Website, Mobile and Customer Transaction Platform

Theory

The platform processes billions of clicks, mobile interactions and customer transactions, so a distributed architecture is required. The probable design uses a combination of distributed storage, event streaming, batch processing and real-time analytics.

1. Distributed Storage

A likely storage architecture is:

- HDFS or cloud object storage such as Amazon S3/Azure Data Lake for large-scale raw data.
- Hive/Delta Lake/Iceberg for structured analytical datasets.
- NoSQL databases such as Cassandra or MongoDB for high-volume semi-structured data.

Structured transaction data may be stored in tables containing customer ID, product ID, price and transaction time. Semi-structured clickstream data may be stored as JSON or similar event records.

2. Data Ingestion

Apache Kafka is a probable ingestion framework because it can handle billions of events through distributed topics and partitions.

3. Batch Processing

Apache Spark or Hadoop MapReduce can process historical data.

Spark is particularly suitable because it supports:

- Large-scale batch processing
- SQL analytics
- Machine learning
- Stream processing

4. Stream Processing

For real-time analytics, the platform may use:

- Apache Flink
- Spark Structured Streaming
- Kafka Streams

These systems process click and transaction events continuously.

5. Partitioning and Scalability

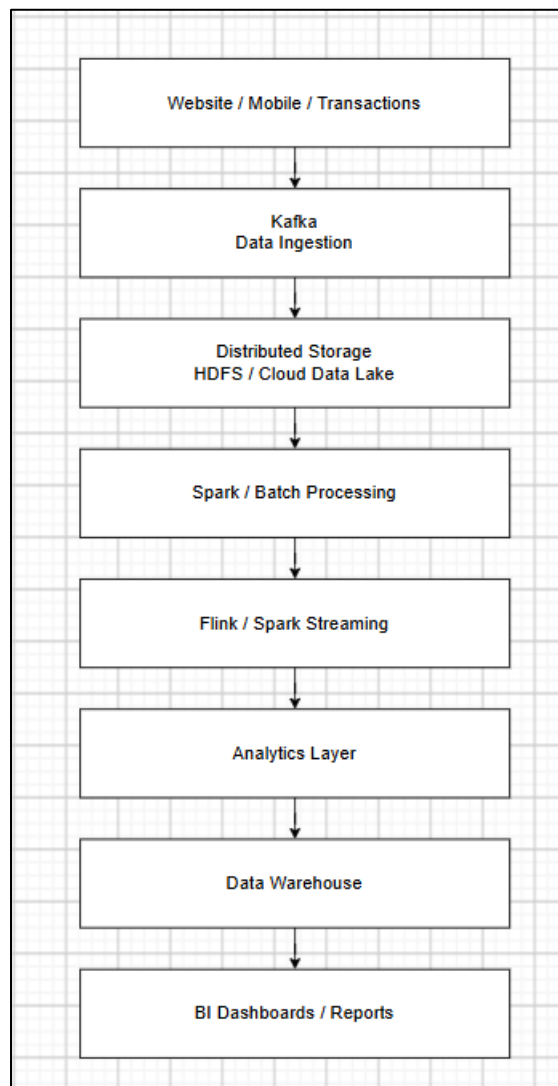
The data is probably partitioned using fields such as:

- Date
- Region
- Customer ID
- Event type

Kafka partitions allow multiple consumers to process events simultaneously. Replication provides fault tolerance if a storage node or processing node fails.

6. Analytics and BI

Processed data is loaded into a data warehouse or analytical database. Business intelligence tools such as Power BI, Tableau or Looker can connect to this analytical layer.



Conclusion

The probable architecture combines Kafka + distributed storage + Spark/Flink + analytical databases + BI tools. Partitioning, replication and horizontal scaling allow the platform to process massive workloads efficiently.

2. E-Commerce Personalization Platform

Theory

The e-commerce platform collects product views, cart additions, abandoned checkouts and payment events. These events can be processed in real time to understand customer intent and generate personalized recommendations.

1. Event Streaming

Kafka is a likely event-streaming platform.

Events may include:

Product View

Cart Addition

Checkout

Payment

Abandoned Cart

Each event can contain customer ID, session ID, product ID, timestamp and region.

2. Customer Profile Database

A NoSQL database such as Cassandra, MongoDB or DynamoDB can maintain rapidly changing customer profiles.

The profile may contain:

- Recent searches
- Viewed products
- Purchased products
- Cart contents
- Preferences

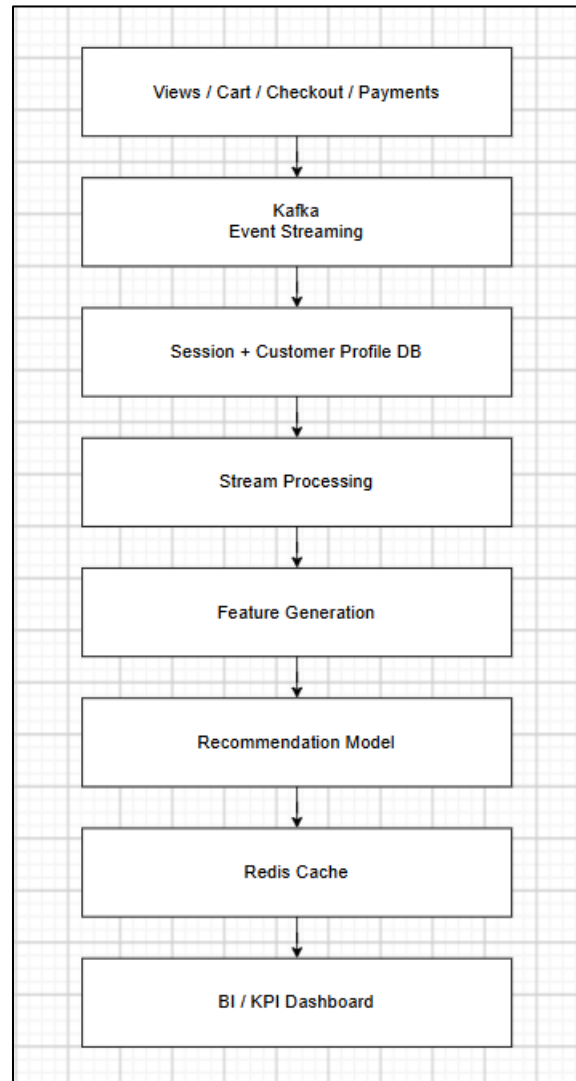
3. Data Synchronization

Session and transaction information can be connected using a common:

- Customer ID
- Session ID
- Product ID
- Timestamp

This allows browsing behavior to be joined with actual orders.

4. Recommendation Workflow



Possible models include:

- Collaborative filtering
- Content-based recommendation
- Hybrid recommendation
- Neural recommendation models

5. Caching

A system such as Redis may cache:

- Customer profiles
- Popular products
- Recent recommendations
- Product information

Caching reduces response time.

6. Distributed Query Processing

Spark SQL, Trino or Presto may be used for querying large datasets stored in a data lake.

7. Structured + Semi-Structured Data

Structured order data can be combined with semi-structured browsing events using customer/session IDs.

8. Fault Tolerance

The system may use:

- Kafka replication
- Distributed storage replication
- Multiple processing nodes
- Automatic task recovery
- Checkpointing

9. KPI Dashboard

Important KPIs include:

- Conversion rate
- Cart abandonment rate
- Average order value
- Customer retention
- Revenue
- Product views

Conclusion

A likely architecture uses Kafka, NoSQL customer profiles, Redis, Spark/Trino and machine learning to provide scalable real-time personalization while maintaining fault tolerance.

3. Big Data Healthcare System

Theory

Healthcare data contains different formats including patient records, diagnostic images, wearable sensor streams and prescription information. Therefore, a hybrid storage architecture is likely required.

1. Hybrid Database Architecture

Structured data can be stored in relational databases:

- PostgreSQL
- MySQL
- Oracle

Examples:

- Patient IDs
- Prescriptions
- Laboratory results
- Appointment records

Unstructured data such as medical images can be stored using:

- Object storage
- HDFS
- Medical image repositories

NoSQL databases can store high-volume sensor and semi-structured information.

2. Data Integration

Healthcare organizations need interoperability between hospitals, laboratories and insurance providers.

Probable technologies include:

- APIs
- ETL pipelines
- HL7
- FHIR
- DICOM for medical images

3. Streaming

Wearable devices generate continuous data such as:

- Heart rate
- Blood pressure
- Oxygen level
- Activity

Kafka can collect these streams, while Flink or Spark Streaming can process them.

4. Machine Learning

ML models may support:

- Disease prediction
- Patient risk scoring
- Readmission prediction
- Abnormal vital-sign detection

Possible algorithms include:

- Logistic Regression
- Random Forest
- XGBoost
- Neural Networks

5. Privacy and Security

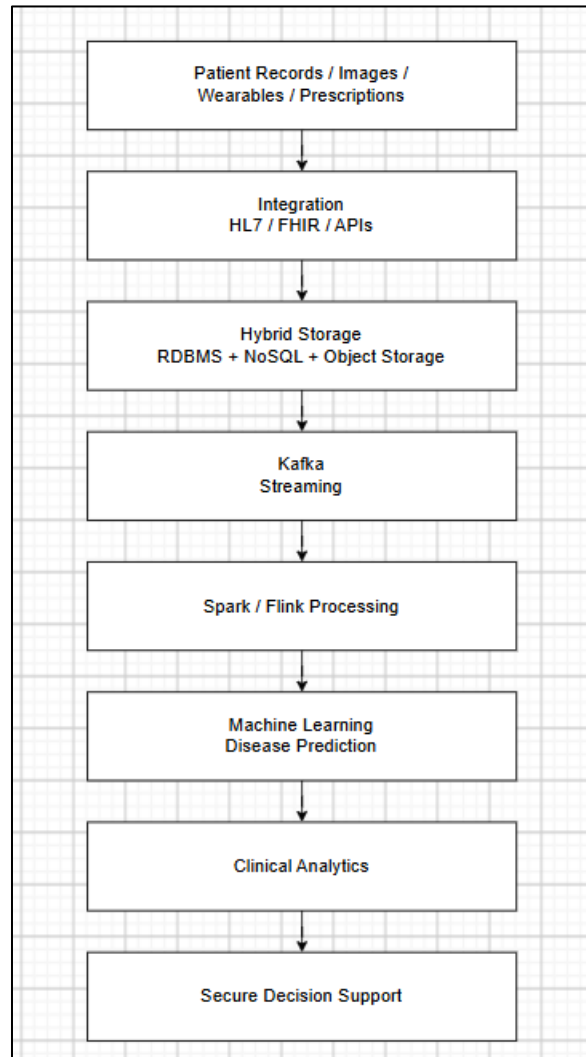
Healthcare information requires strong protection.

Likely controls include:

- Encryption at rest
- Encryption during transmission
- Role-based access control
- Authentication
- Audit logging
- Data masking
- Data anonymization/pseudonymization where appropriate

Compliance requirements depend on the country and healthcare context.

6. Clinical Analytics Pipeline



7. Population Health

Aggregated and appropriately de-identified data can be processed using **Spark** to identify:

- Disease trends
- Population risk
- Treatment outcomes
- Healthcare utilization patterns

Conclusion

A probable healthcare architecture combines relational databases + object storage + NoSQL + Kafka + Spark/Flink + ML, with strong privacy, encryption and auditing mechanisms.

4. Smart City Big Data Platform

Theory

A smart city continuously receives data from traffic sensors, CCTV metadata, weather systems and public transportation. Since some decisions must be made immediately, the architecture needs both edge computing and centralized cloud/big-data processing.

1. Edge Computing

Edge devices can process data close to where it is generated. This reduces bandwidth usage and latency.

2. Distributed Messaging

Apache Kafka can collect continuous events from: Traffic sensors, Public transport, Weather systems, Edge devices

3. Central Storage

Long-term data can be stored in: Cloud object storage, Data lakes, HDFS

4. Time-Series Database

Traffic and sensor measurements are time-dependent. Suitable databases include: InfluxDB, TimescaleDB

These are useful for time-based queries.

5. Geospatial Database

For location-based queries, PostGIS/PostgreSQL or other geospatial databases can be used.

6. Stream Processing

Flink or Spark Streaming can process traffic events continuously.

Possible analytics:

- Traffic congestion detection
- Accident detection
- Travel-time prediction
- Public transport monitoring

7. Batch + Real-Time Analytics

Real-time analytics supports immediate operational decisions, while batch analytics helps long-term planning.

8. Security

Likely security measures include:

- Encryption
- Authentication, Role-based access
- Network segmentation, API security

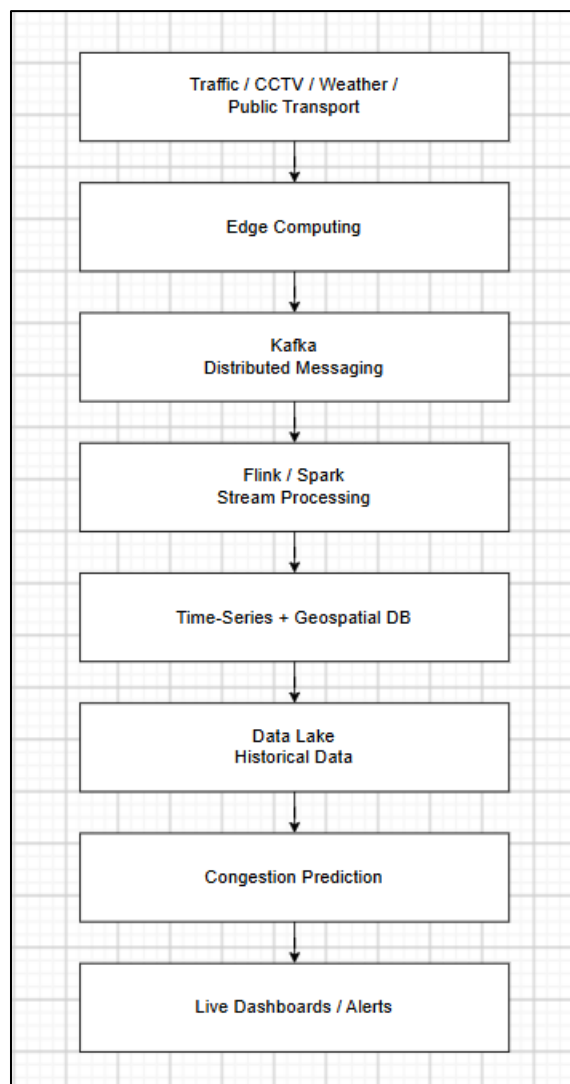
- Audit logs, Monitoring

CCTV and citizen-related information should have strict access restrictions.

9. Visualization

Tools such as Power BI, Tableau or Grafana can provide live dashboards showing:

- Traffic congestion
- Vehicle movement
- Weather conditions
- Public transport status
- Alerts



Conclusion

A smart-city platform would likely combine edge computing, Kafka, stream processing, cloud storage, time-series/geospatial databases and BI dashboards to provide real-time operational intelligence and long-term urban planning insights.

5. Healthcare Analytics System

Theory

This system integrates patient records, wearable readings, laboratory reports and appointment histories. Because medical data is both structured and unstructured, a hybrid storage architecture is appropriate.

1. Hybrid Storage

Relational databases store structured information such as: Patient details, Appointments, Lab results, Prescriptions

Object storage/data lakes store: Medical documents, Images, Reports

NoSQL databases can store high-volume wearable readings.

2. ETL and Interoperability

ETL pipelines extract information from hospitals and clinics, transform it into standardized formats and load it into the analytics platform.

Interoperability can use standards such as: HL7, FHIR, DICOM for medical imaging

3. Real-Time Monitoring

Wearable data can be streamed through Kafka.

4. Predictive Analytics

Historical medical information can be used to train machine learning models.

Models may predict:

- Disease risk
- Patient deterioration
- Readmission risk
- Treatment response

5. Distributed Processing

Population-level healthcare datasets can contain millions of patient records. Apache Spark can divide the workload across multiple nodes and process these datasets efficiently.

6. Data Governance

Security mechanisms include:

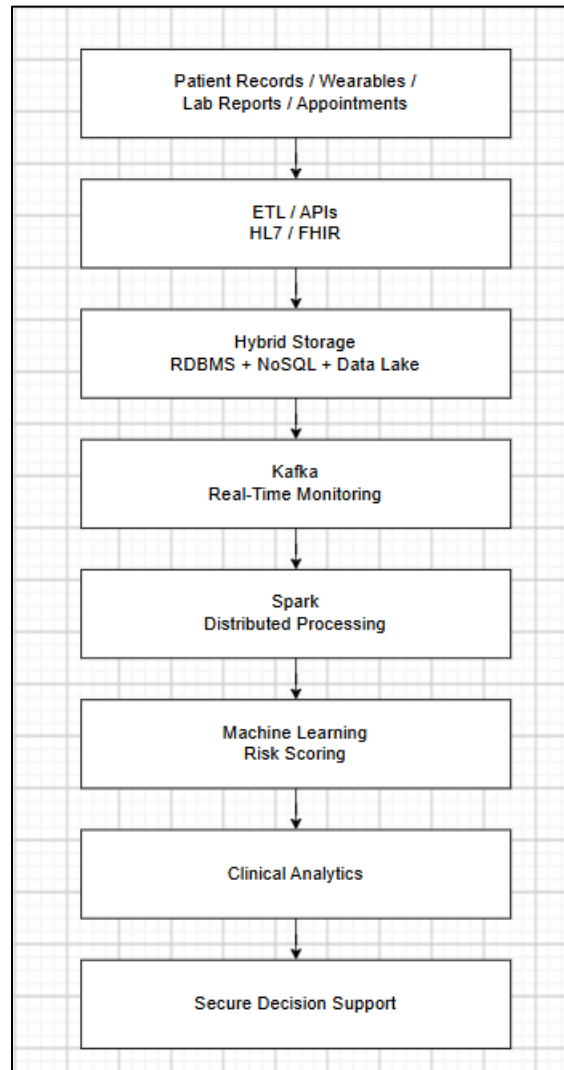
- Encryption at rest and in transit, Strong authentication
- Role-based access, Audit trails
- Data masking, Data retention policies
- Access monitoring

Applicable healthcare privacy regulations must be followed based on the jurisdiction.

7. Machine Learning

ML models can calculate a risk score from patient features.

Treatment recommendations should be treated as decision-support outputs, with appropriate clinical validation and human oversight rather than fully autonomous decisions.



Conclusion

A hybrid healthcare analytics architecture using relational databases, NoSQL, cloud data lakes, Kafka, Spark/Flink and machine learning can support both real-time monitoring and large-scale population analytics while maintaining strong security and governance.